

Лекція 7. GitHub Actions

GitHub Actions був вперше анонсований у жовтні 2018 року на конференції GitHub Universe. Цей анонс викликав значний інтерес у спільноті розробників завдяки новій можливості автоматизувати робочі процеси безпосередньо в екосистемі GitHub. Після періоду бета-тестування інструмент був офіційно запусканий у листопаді 2019 року.

GitHub Actions — це вбудований сервіс GitHub для автоматизації робочих процесів (workflow) безпосередньо у репозиторії. Простіше кажучи, GitHub Actions дозволяє автоматизувати завдання, які виконуються у відповідь на певні події в репозиторії

Мета лекції:

- Ознайомити з історією виникнення GitHub Actions.
- Розглянути роботу інструменту.
- Порівняти з іншими системами автоматизації.
- Показати приклади використання.



GitHub Actions

Головною метою створення GitHub Actions було спрощення процесу автоматизації для розробників, які вже активно використовують GitHub для управління репозиторіями. На той час багато компаній і команд використовували сторонні CI/CD сервіси, такі як Jenkins, Travis CI чи CircleCI, які вимагали додаткового налаштування та інтеграції. GitHub Actions став логічним рішенням, що об'єднало функції CI/CD із можливістю створення універсальних автоматизованих сценаріїв.

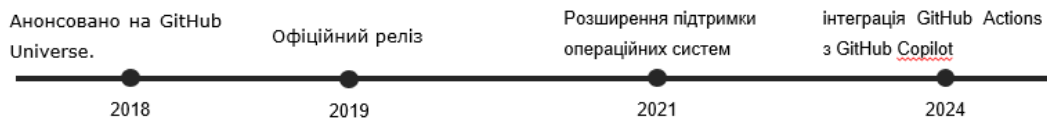
З розвитком DevOps та практик CI/CD дедалі більше команд потребували інструментів, які б допомагали знижувати кількість рутинних завдань і прискорювали процеси розробки та доставки програмного забезпечення. Автоматизація стала критично важливою, особливо в умовах сучасного динамічного ринку програмних продуктів.

GitHub, як лідер серед платформ для управління кодом, стратегічно вирішив інтегрувати автоматизацію безпосередньо у свою екосистему. Це дозволило

розробникам уникати використання сторонніх сервісів для CI/CD і спростило процес інтеграції робочих процесів із репозиторіями.

GitHub Actions створювався з акцентом на зручність і простоту, зменшуючи поріг входу для автоматизації навіть для команд, які не мали значного досвіду роботи з CI/CD інструментами. Використання декларативного формату YAML для опису робочих процесів зробило налаштування інтуїтивно зрозумілим і доступним.

Історія виникнення GitHub Actions



- Мета: надати вбудовану CI/CD платформу для проектів на GitHub.
- Заміна зовнішніх систем (Jenkins, Travis CI) інтегрованим рішенням.

Ще одним важливим фактором стала гнучкість інструменту. GitHub Actions запропонував широку підтримку для різних операційних систем (Windows, MacOS, Linux) та мов програмування, що зробило його універсальним рішенням для багатьох проєктів. Він став інструментом, який можна адаптувати під різноманітні сценарії, від простих автоматизаційних задач до складних багатоступеневих робочих процесів.

Загалом, GitHub Actions був створений як відповідь на запит індустрії на зручний, потужний і інтегрований інструмент автоматизації, що дозволяє командам будь-якого рівня підготовки швидко налаштовувати процеси CI/CD та покращувати ефективність розробки.

З моменту свого запуску GitHub Actions став основним інструментом для автоматизації робочих процесів у рамках GitHub. Завдяки постійному розвитку і впровадженню нових можливостей, він активно використовується як малими

командами, так і великими корпораціями, здобувши популярність серед спільнот DevOps, CI/CD та open-source розробників.

Що таке GitHub Actions?

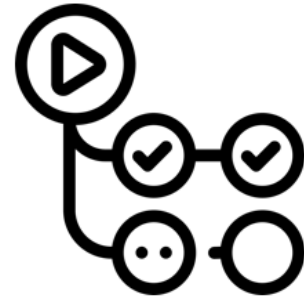
Workflow — набір автоматизованих дій, що реагує на події в репозиторії.

Job — окрема задача в процесі workflow.

Step — крок, що виконує конкретну дію.

Action — готова функція, яку можна використовувати в Step.

Runner — сервер, де виконуються завдання.



GitHub Actions став вагомим фактором у трансформації автоматизації процесів у сфері розробки програмного забезпечення. Інструмент не лише спростив впровадження CI/CD для розробників, але й створив нові можливості для вдосконалення робочих процесів.

GitHub Actions зробив автоматизацію доступною навіть для невеликих команд і окремих розробників. Завдяки інтеграції з екосистемою GitHub, командам більше не потрібно витрачати ресурси на налаштування складної інфраструктури для CI/CD. Відсутність необхідності у використанні додаткового програмного забезпечення дозволила багатьом проектам швидко перейти до автоматизації. Безкоштовний доступ для публічних репозиторіїв зробив інструмент ідеальним вибором для open-source спільноти.

GitHub Actions сприяв стандартизації автоматизації за допомогою опису робочих процесів у форматі YAML. Ця декларативна структура забезпечує прозорість і легкість у підтримці. Вона також знизилася бар'єр входу для новачків, підвищивши сумісність і сприяючи обміну знаннями між командами.

Інтеграція GitHub Actions із GitHub Marketplace стала ключовим моментом у розвитку екосистеми. Спільнота розробників створила тисячі готових дій (actions), які значно прискорюють налаштування автоматизації. Marketplace став

платформою для поширення рішень, що забезпечує легке впровадження популярних інструментів та сервісів, таких як Docker, AWS та Kubernetes.

Інструмент активно сприяє поширенню DevOps-підходу, об'єднуючи розробників та операційні команди. Автоматизація тестування, розгортання та моніторингу дозволила командам зосередитися на вдосконаленні продуктів, залишаючи рутинні завдання автоматизації. Навіть малі команди, які не мають доступу до складних рішень, отримали можливість впроваджувати DevOps-практики завдяки GitHub Actions.

З появою цього інструменту ринок CI/CD систем зазнав значних змін. Конкуренти, такі як Jenkins, GitLab CI/CD і CircleCI, почали адаптувати свої рішення, щоб відповідати новим вимогам спрощення налаштувань та інтеграції. Зростання популярності GitHub Actions стимулювало здорову конкуренцію, що позитивно вплинуло на розвиток індустрії загалом.

GitHub Actions став важливим інструментом для open-source спільноти. Розробники отримали платформу, яка дозволяє легко налаштовувати тестування та релізи для публічних проєктів без витрат. Багато популярних open-source проєктів перейшли на використання GitHub Actions, що сприяло подальшому вдосконаленню цього інструменту.

Крім того, GitHub Actions став платформою для інновацій у робочих процесах. Інтеграція з інструментами машинного навчання та штучного інтелекту відкрила нові можливості для автоматизації аналізу даних та оптимізації процесів. Кастомні дії дозволяють створювати унікальні рішення під конкретні потреби бізнесу, сприяючи гнучкості та адаптивності.

GitHub Actions змінив правила гри на ринку автоматизації, зробивши CI/CD доступним для всіх. Його вплив поширюється від малих команд до великих підприємств, стимулюючи розвиток DevOps-практик, стандартизацію процесів і впровадження інновацій у розробці програмного забезпечення.

Розбір інструменту GitHub Actions

GitHub Actions – це потужний інструмент автоматизації, який забезпечує розробників широкими можливостями для створення, тестування, розгортання

додатків і виконання інших завдань безпосередньо у межах екосистеми GitHub. Він побудований на ідеї інтеграції та простоти, надаючи як базові, так і розширені функції для автоматизації робочих процесів.

Основні можливості GitHub Actions

Автоматизація процесів

GitHub Actions дозволяє автоматизувати широкий спектр завдань, включаючи:

- *запуск тестів*: автоматичне тестування змін у коді, що забезпечує стабільність проєкту;
- *розгортання додатків*: інструмент дозволяє безперервно розгортати додатки на хмарних сервісах, серверах або контейнерах;
- *управління релізами*: генерація релізів на основі певних подій, таких як створення нової гілки або тегу;
- *моніторинг і повідомлення*: надсилання сповіщень у Slack, Telegram або інші сервіси про успішне виконання чи помилки;
- *інтеграція сторонніх сервісів*: легка інтеграція з такими сервісами, як Docker, Kubernetes, AWS, Azure, Google Cloud та іншими.

Декларативний підхід

GitHub Actions використовує декларативний підхід для створення робочих процесів, що дозволяє чітко та зрозуміло описати автоматизовані завдання. Основна ідея цього підходу полягає в тому, що замість визначення процедур для кожного кроку, користувач описує самі результати і поведінку робочих процесів. Це знижує складність і робить налаштування автоматизації більш доступним для розробників, навіть без глибоких технічних знань.

Основні елементи декларативного підходу в GitHub Actions:

- *формат YAML*: GitHub Actions використовує формат YAML для опису робочих процесів. YAML є простим та інтуїтивно зрозумілим, що дозволяє навіть новачкам легко писати та підтримувати сценарії. Завдяки цьому

формату можна ефективно описувати навіть складні робочі процеси без необхідності в складному програмуванні;

- *логічна структура*: робочий процес (Workflow) складається з кількох основних компонентів:
 - Events (події): це тригери, що запускають робочий процес, наприклад, push, pull request, issue тощо;
 - Jobs (завдання): завдання – це набір кроків (steps), які виконуються на певному середовищі (наприклад, Ubuntu, Windows чи MacOS);
 - Steps (кроки): це окремі операції в рамках завдання, такі як виконання скриптів або запуск дій.

Ця чітка і зрозуміла структура допомагає організувати робочі процеси, знижує ймовірність помилок і спрощує подальше обслуговування автоматизованих сценаріїв. Користувачі можуть легко управляти та розширювати свої процеси, додаючи нові події, завдання або кроки, що робить систему надзвичайно гнучкою.

Інтеграція з репозиторієм

GitHub Actions забезпечує тісну інтеграцію з репозиторіями GitHub, що дозволяє автоматично виконувати завдання на основі подій, які відбуваються в репозиторії. Це дає змогу створювати гнучкі та потужні робочі процеси, які активуються безпосередньо від подій, що відбуваються в репозиторії, без необхідності в зовнішніх тригерах чи інструментах.

Основні події, які активують робочий процес (workflow):

- *push*: ця подія спрацьовує, коли код змінюється в певній гілці репозиторію. Це дозволяє запускати автоматичні тести або розгортання при кожній зміні в коді;
- *pull_request*: спрацьовує, коли створюється або оновлюється pull request, що дозволяє виконувати перевірки, тестування або інші дії до злиття змін у головну гілку;

- *schedule*: подія, яка активує робочий процес на основі cron-розкладу. Це корисно для виконання періодичних завдань, таких як оновлення залежностей, регулярне тестування чи резервне копіювання;
- *issue*: відкриття або закриття завдання (issue) може запустити робочий процес, наприклад, для автоматизації коментарів або керування проектними задачами;
- *release*: спрацьовує, коли створюється новий реліз, що дозволяє автоматично виконувати завдання, такі як створення changelog або оновлення документації.

GitHub Actions має доступ до коду, історії комітів та інших метаданих репозиторіїв GitHub. Крім того, інструмент дозволяє працювати з GitHub API для автоматизації різних дій. Наприклад, можна автоматично створювати коментарі до pull request, оновлювати статуси завдань чи виконувати інші інтеракції з репозиторієм, що значно розширює можливості для автоматизації та покращення взаємодії в команді.

Гнучкість і кастомізація

GitHub Actions підтримує як використання готових дій, так і створення кастомних рішень:

- *готові дії (Reusable Actions)*:
 - у GitHub Marketplace доступні тисячі готових дій для різних завдань, таких як запуск тестів, розгортання контейнерів або інтеграція зі сторонніми сервісами;
 - приклад використання дії для налаштування Node.js:

```
yaml
- uses: actions/setup-node@v3
  with:
    node-version: '16'
```

- *кастомні дії*: розробники можуть створювати власні дії на основі скриптів або Docker-контейнерів:
 - скриптові дії: написані на будь-якій мові програмування (наприклад, JavaScript);
 - дії на основі контейнерів: забезпечують незалежність середовища виконання;
 - матричні тести: можливість паралельного виконання завдань у різних середовищах або з різними параметрами.

yaml

```
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest]
    node: [14, 16]
```

Ключові компоненти GitHub Actions

GitHub Actions складається з кількох основних елементів, які дозволяють гнучко налаштовувати робочі процеси автоматизації. Завдяки цим компонентам розробники можуть легко адаптувати інструмент для виконання різноманітних завдань.

Workflow (робочий процес)

Структура Workflow

Workflow визначається у файлі YAML у `.github/workflows/`.
Основні елементи YAML:

- *on*: — подія, що запускає сценарій.
- *jobs*: — набір завдань для виконання.
- *steps*: — дії в межах завдання.

Можливість використання готових Action з GitHub Marketplace.

Приклад простого Workflow

```
1 name: CI Workflow
2 on: [push, pull_request]
3
4 jobs:
5   build:
6     runs-on: ubuntu-latest
7     steps:
8       - uses: actions/checkout@v4
9       - name: Install dependencies
10         run: npm install
11       - name: Run tests
12         run: npm test
```


Workflow – це набір інструкцій, які визначають, як і коли виконується автоматизація. Вони описуються у файлах формату YAML, що зберігаються в каталозі `.github/workflows` репозиторія.

Основні особливості:

- один репозиторій може містити кілька workflow, кожен із яких відповідає за виконання певних завдань (наприклад, CI, деплоймент або моніторинг);
- Workflow можуть бути простими (один Job) або складними (кілька взаємозалежних Job);
- вони активуються певними подіями (Events).

Приклад мінімального workflow:

```
yml
name: CI Workflow
on: push
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: echo "Hello, world!"
```

Event (подія)

Event – це тригер, що запускає виконання workflow. Події визначаються у розділі `on` у YAML-файлі.

Типи подій:

- *події GitHub*:
 - *push*: коли зміни надсилаються до певної гілки;
 - *pull_request*: коли створюється або оновлюється pull request;
 - *release*: коли створюється новий реліз;
 - *issue*: коли створюється або оновлюється issue.

- *заплановані події*: виконання завдань за розкладом за допомогою синтаксису Cron (schedule);
- *ручний запуск*: використання події `workflow_dispatch` для запуску workflow вручну.

Приклад запланованого запуску:

```
yaml
on:
  schedule:
    - cron: "0 0 * * *" # Щодня опівночі
```

Job (завдання)

Job – це набір кроків (Steps), які виконуються у певному середовищі (Runner). Завдання у workflow можуть бути виконані паралельно або послідовно.

Особливості завдань:

- завдання визначаються у розділі jobs;
- кожне завдання має свою платформу виконання (runs-on), наприклад, ubuntu-latest, windows-latest або macos-latest;
- завдання можуть залежати одне від одного через параметр needs.

Приклад завдань із залежностями:

```
yaml

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Build process"
  test:
    runs-on: ubuntu-latest
    needs: build
    steps:
      - run: echo "Testing process"
```

Step (крок)

Step – це окремий крок у межах Job, який виконує конкретну дію, наприклад:

- завантаження коду репозиторія;
- запуск скрипта;
- Виклик дії (Action);
- Типи кроків:
 - дії (Actions): використання готових або кастомних модулів;
 - команди оболонки (Shell Commands): виконання команд у середовищі виконання.

Приклад кроків:

```
yaml

steps:
  - name: Checkout code
    uses: actions/checkout@v3
  - name: Install dependencies
    run: npm install
  - name: Run tests
    run: npm test
```

Actions (дії)

Actions – це окремі виконувані модулі, які дозволяють виконувати повторювані завдання. Дії можуть бути створені розробниками або використані з GitHub Marketplace.

Типи дій:

- *готові дії*: наприклад, actions/checkout для завантаження коду репозиторія або actions/setup-node для налаштування середовища Node.js;
- *кастомні дії*: створюються розробниками для вирішення специфічних завдань.

Приклад використання готової дії:

```
yaml
- name: Checkout code
  uses: actions/checkout@v3
```

Приклад створення кастомної дії (на основі JavaScript):

Код дії:

```
javascript

const core = require('@actions/core');
const github = require('@actions/github');

try {
  const name = core.getInput('name');
  console.log(`Hello, ${name}!`);
} catch (error) {
  core.setFailed(error.message);
}
```

Використання кастомної дії:

```
yaml

steps:
  - uses: ./my-action
    with:
      name: 'World'
```

GitHub Actions надає гнучку архітектуру, яка дозволяє легко адаптувати її до потреб конкретного проєкту. Кожен із компонентів доповнює інші, забезпечуючи можливість створення простих або складних автоматизованих сценаріїв.

Відмінності GitHub Actions від аналогічних систем

Параметр	GitHub Actions	Jenkins	GitLab CI/CD	CircleCI	Travis CI
Інтеграція	Глибока з GitHub	Плагіни потрібні	Глибока з GitLab	Вимагає налаштувань	Вимагає налаштувань
Простота	Висока	Низька	Середня	Висока	Висока
Готові рішення	GitHub Marketplace	Плагіни	Обмежені	Обмежені	Обмежені
Цінова політика	Безкоштовно для публічних	Безкоштовний (але хостинг)	Безкоштовно для публічних	Безкоштовний рівень	Обмежено безкоштовний
Гнучкість	Висока для більшості задач	Дуже висока	Висока	Середня	Середня